

Article

Energy-Efficient Dual-Core RISC-V Architecture for Edge AI Acceleration with Dynamic MAC Unit Reuse

Cristian Andy Tanase 

Faculty of Electrical Engineering and Computer Science, Stefan cel Mare University of Suceava,
720229 Suceava, Romania; andy.tanase@usm.ro

Abstract

This paper presents a dual-core RISC-V architecture designed for energy-efficient AI acceleration at the edge, featuring dynamic MAC unit sharing, frequency scaling (DFS), and FIFO-based resource arbitration. The system comprises two RISC-V cores that compete for shared computational resources—a single Multiply–Accumulate (MAC) unit and a shared external memory subsystem—governed by a channel-based arbitration mechanism with CPU-priority semantics, while each core maintains private instruction and data caches. The architecture implements a tightly coupled Neural Processing Unit (NPU) with CONV, GEMM, and POOL operations that execute opportunistically in the background when the MAC unit is available. Dynamic frequency scaling (DFS) with three levels (100/200/400 MHz) is applied to the shared MAC unit, allowing the dynamic acceleration of CNN workloads. The arbitration mechanism uses SystemC `sc_fifo` channels with CPU-priority polling, ensuring that CPU execution is minimally impacted by background AI processing while the NPU makes progress during idle MAC slots. The NPU supports 3×3 convolutions, matrix multiplication (GEMM) with 10×10 tiles, and pooling operations. The implementation is cycle-accurate in SystemC, targeting FPGA deployment. Experimental evaluation demonstrates that the dual-core architecture achieves $1.87\times$ speedup with 93.5% efficiency for parallel workloads, while DFS enables 70% power reduction at low frequency. The system successfully executes simultaneous CPU and AI workloads, with CPU-priority arbitration ensuring no CPU starvation under contention. The proposed design offers a practical solution for embedded AI applications requiring both general-purpose computation and neural network acceleration, validated through comprehensive SystemC simulation on modern FPGA platforms.

Keywords: RISC-V; dual core; memory arbitration; DFS; MAC sharing; CNN acceleration; CONV; GEMM; POOL; energy-efficient AI; edge computing; SystemC; HLS; FPGA



Academic Editors: Taskin Kocak and
Ron (Rongyu) Lin

Received: 28 January 2026

Revised: 21 March 2026

Accepted: 26 March 2026

Published: 1 April 2026

Copyright: © 2026 by the author.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Convolutional Neural Networks (CNNs) have become ubiquitous in modern AI applications, but their efficient deployment on embedded systems remains a critical challenge due to strict constraints on power consumption, memory bandwidth, and latency. Deploying inference workloads on FPGA-based systems or integrated System-on-Chip (SoC) processors requires careful architectural decisions that balance performance gains against the overhead of specialized hardware.

Conventional CPU-NPU architectures allocate dedicated Multiply–Accumulate (MAC) units exclusively to the neural accelerator, separate from those in the main processor. This approach necessarily increases silicon area, power dissipation, and design complexity.

However, in many real-world workloads, the MAC unit of the CPU experiences significant idle periods, particularly during non-arithmetic instructions and cache misses. This observation motivates a novel co-design paradigm: dynamic MAC sharing.

This paper presents a dual-core RISC-V system with a tightly integrated Neural Processing Unit (NPU) that shares the MAC unit opportunistically with the main processors. The system features:

1. Dual-core RISC-V processors that execute general-purpose workloads in parallel, each with independent instruction fetch and decode stages but sharing the data cache, instruction cache, external memory, and a single MAC unit.
2. CPU-priority arbitration with parity-based fairness that enforces fair (50–50%) access to shared resources between CPU0 and CPU1, while the NPU operates in best-effort mode, utilizing idle MAC slots without delaying CPU execution.
3. Dynamic frequency scaling (DFS) applied to the NPU compute tiles, enabling the dynamic acceleration of CNN workloads to match RISC-V throughput and prevent the accelerator from bottlenecking the dual-core processors.
4. Opportunistic AI acceleration via an NPU executing CNN operations (CONV, GEMM, and POOL) in the background when the MAC unit is available, without interrupting or blocking CPU execution.

The dual-core architecture provides inherent parallelism for mixed CPU-AI workloads, while the CPU-priority arbitration mechanism ensures predictable, non-starving performance for both RISC-V cores (50–50% fairness between CPU0 and CPU1). The NPU operates opportunistically, receiving MAC grants only when both CPUs are idle, ensuring background AI processing never delays general-purpose execution. DFS on the MAC unit dynamically adjusts frequency to match workload intensity.

The NPU implements three specialized compute tiles:

- CONV Tile: 2D convolution operations with configurable kernel sizes (3×3 , 5×5), stride, padding, and multi-channel support.
- GEMM Tile: General matrix multiplication for fully connected layers and linear transformations, with tiling strategy to reduce memory pressure.
- POOL Tile: Max and average pooling operations for spatial downsampling and feature map reduction.

The architecture is implemented in cycle-accurate SystemC, targeting FPGA synthesis and supporting end-to-end CNN inference simulation. The FIFO-based channel architecture ensures that each requestor (CPU Core 0, CPU Core 1, and NPU) can access the shared MAC unit and memory subsystem, with CPU operations receiving priority.

The model is fully synthesizable and was validated for hardware compatibility using Vivado HLS 2019. However, this work focuses on architectural exploration and cycle-accurate simulation, with post-synthesis resource and power estimates. Full FPGA deployment and measurement are left for future work due to time and resource constraints. All simulations were performed using SystemC 2.3.3 on Linux, using g++ compiler.

The CPU core used in this work is based on the HL5 RISC-V model [1], which was extensively modified at all pipeline stages to interface with the NPU, support dual-core arbitration, and implement custom CSR registers for AI configuration. The NPU accelerator and the DFS mechanism were designed and implemented entirely from scratch for this architecture, and integrated tightly with the modified CPU pipeline.

1.1. Motivation and Problem Statement

Deploying CNN inference on resource-constrained embedded platforms presents a trade-off dilemma:

- Dedicated accelerators (e.g., Gemmini, DianNao, and Eyeriss) achieve high throughput but require separate MAC arrays, caches, and memory ports, incurring prohibitive area and power overheads on small FPGA platforms or edge devices.
- Pure software execution on a scalar CPU is power efficient but suffers from low throughput, limiting the real-time inference capabilities.
- Dual-core processors are common in modern SoCs but typically implement independent MAC units per core, further increasing hardware complexity.

This work addresses these challenges by introducing dynamic MAC sharing, where the two CPU cores share a single MAC unit under parity-based 50–50% fairness, while the NPU competes opportunistically for idle slots. Additionally, dual-core parallelism and DFS enable the simultaneous execution of general-purpose and AI workloads with energy-efficient power scaling.

1.2. Main Contributions

The primary contributions of this paper are:

1. CPU-priority arbitration protocol that guarantees 50–50% fair access to shared resources (MAC, I-cache, D-cache, and memory) between the two RISC-V cores, with the NPU operating in best-effort mode during idle MAC slots, ensuring CPU execution is never delayed by background AI processing.
2. Dual-core RISC-V architecture with shared MAC unit, demonstrating near-linear speedup for parallel workloads and effective resource utilization through dynamic sharing.
3. DFS on NPU tiles enabling dynamic frequency scaling of the AI accelerator, preventing CNN computation from becoming a bottleneck while maintaining energy efficiency compared to dedicated high-frequency accelerators. Note that on FPGA platforms, voltage scaling is not directly feasible; only the clock frequency is adjusted dynamically.
4. Opportunistic NPU architecture (CONV, GEMM, and POOL tiles) that executes in the background without blocking CPU execution or introducing timing side-channels.
5. Cycle-accurate SystemC implementation with comprehensive simulation of mixed CPU-AI workloads, validated on modern FPGA platforms.
6. Empirical evaluation showing: (1) CPU0 and CPU1 each receive approximately 41% of MAC grants under contention, confirming fair dual-core access; (2) NPU receives the remaining 18% during idle slots without delaying CPU execution; (3) DFS reduces NPU power by up to 70% at lower frequencies while maintaining acceptable inference latency.

1.3. Paper Organization

Section 2 reviews related work. Section 3 presents the overall system architecture. Section 4 describes the specialized NPU compute tiles: CONV, GEMM, and POOL. Section 5 reports experimental results and performance evaluation. Section 6 provides discussion. Section 7 concludes the paper.

2. Related Work

Hardware acceleration of neural networks has been intensively studied over the past decade, both in dedicated NPU architectures and in heterogeneous CPU–accelerator systems. Early approaches focused on fully specialized architectures for CNN acceleration on FPGAs, exploiting massive parallelism at the MAC level and memory optimizations. Classical works by Zhang et al. [2], Qiu et al. [3], and Venieris and Bouganis [4] demonstrated

the potential of FPGAs for CNN acceleration, but they rely on dedicated MAC units that are not reusable by the CPU.

Beyond CNN-specific accelerators, recent literature reports several FPGA-focused designs relevant to energy-aware and resource-constrained embedded computing, including a convolutional 2D filtering architecture [5], multicore hardware/software co-design on FPGA [6], and run-time adaptation for power-budget variation and fault mitigation in FPGA-based SoPCs [7].

Parametrizable accelerator designs, such as Eyeriss [8] and DianNao [9], introduced concepts like optimized dataflow patterns (row-stationary, weight-stationary) and hierarchical memory architectures to reduce access costs. Although efficient, these architectures assume complete separation of resources between the accelerator and CPU, which increases silicon area and power dissipation.

For embedded platforms, recent work has explored integrating accelerators with scalar processors through AXI interfaces, coprocessors, or custom instructions. RISC-V, with its extensible ISA [10], has been frequently used in such experiments, and generator-based SoCs such as Rocket Chip [11] provide a practical substrate for accelerator integration. However, these designs typically employ separate, dedicated compute units for the accelerator.

Dynamic resource arbitration and opportunistic background execution are less common in the accelerator literature. Open-source accelerators and stacks such as Gemini [12,13], VTA [14], and NVDLA [15] generally assume a dedicated accelerator datapath and do not share the CPU's scalar MAC pipeline with background CNN execution. Likewise, efficient RISC-V inference frameworks such as XpulpNN [16] focus on optimized kernels and ISA-level efficiency, rather than transparent time-sharing of a single MAC unit between CPU and NPU.

At the algorithmic level, convolution can be accelerated using Winograd transforms [17] or FFT-based methods [18]. At the architecture level, prior accelerators emphasize dataflow and memory hierarchy optimizations (e.g., Eyeriss [8] and DianNao/DaDianNao [9,19]), while Bit Fusion [20] explores bit-level composability for efficiency. These approaches generally assume dedicated accelerator hardware rather than transparent CPU-accelerator MAC sharing.

In the RISC-V + AI co-design space, open-source stacks provide reusable baselines, including TVM's VTA [14] and NVDLA [15]. None of these works, however, implement an opportunistic NPU that time-shares the scalar MAC unit with the CPUs under a fairness-aware arbiter.

In contrast to all these works, the architecture proposed in this paper introduces several novel contributions:

1. **Dynamic MAC unit sharing:** Reuse of the existing scalar MAC unit in the RISC-V pipeline, without duplicating the hardware resource and without software interruptions, enabling both CPU and NPU to access the same computational resource under fair arbitration.
2. **Opportunistic background CNN execution:** Three specialized NPU tiles (CONV, GEMM, and POOL) execute in the background when the MAC unit is available, without blocking or stalling the main processors.
3. **Parity-based deterministic arbitration:** A hardware-enforced fair (50–50%) arbitration protocol for shared resources (MAC unit, I-cache, D-cache, and external memory) that guarantees no starvation and maintains predictable performance for all requestors (CPU0, CPU1, and NPU).
4. **Dual-core parallelism with shared resources:** Two independent RISC-V cores executing general-purpose workloads in parallel while competing fairly for shared hardware resources, avoiding the need for separate MAC units per core.

5. DFS for NPU acceleration: Dynamic frequency scaling applied to the AI accelerator tiles to prevent CNN computation from becoming a bottleneck, ensuring the NPU maintains throughput parity with the dual-core RISC-V processors.
6. Transparent hardware integration: Memory-mapped register interface with minimal software overhead, requiring no ISA extensions or interrupt handlers, making the architecture easily portable across RISC-V implementations.

This comprehensive approach significantly reduces hardware cost and increases the effective utilization of shared resources, while DFS prevents the accelerator from limiting overall system performance. The architecture offers an energy-efficient, scalable solution suitable for real-time edge AI applications on FPGA and embedded SoC platforms.

3. System Architecture

The proposed architecture consists of a dual-core RISC-V processor system with a tightly coupled Neural Processing Unit (NPU) that opportunistically shares a single Multiply–Accumulate (MAC) unit. The system is designed such that CNN execution on the NPU occurs in the background without interrupting the main program flow on the RISC-V processors. This approach maximizes MAC utilization, avoids hardware resource duplication, and enables the efficient coexistence of CPU and NPU workloads in a compact architecture suitable for educational FPGA platforms and embedded systems.

3.1. System Overview

Figure 1 provides a high-level view of the system architecture. The platform consists of two independent RISC-V scalar processors (CPU0 and CPU1), each with its own instruction fetch and decode logic and private I-cache/D-cache instances, that execute general-purpose workloads in parallel while sharing a common external memory subsystem (IMEM/DMEM) and a single Multiply–Accumulate (MAC) functional unit. The shared MAC service is accessed by both CPU cores and the NPU through a deterministic hardware arbiter, eliminating the need for dedicated per-core MAC resources.

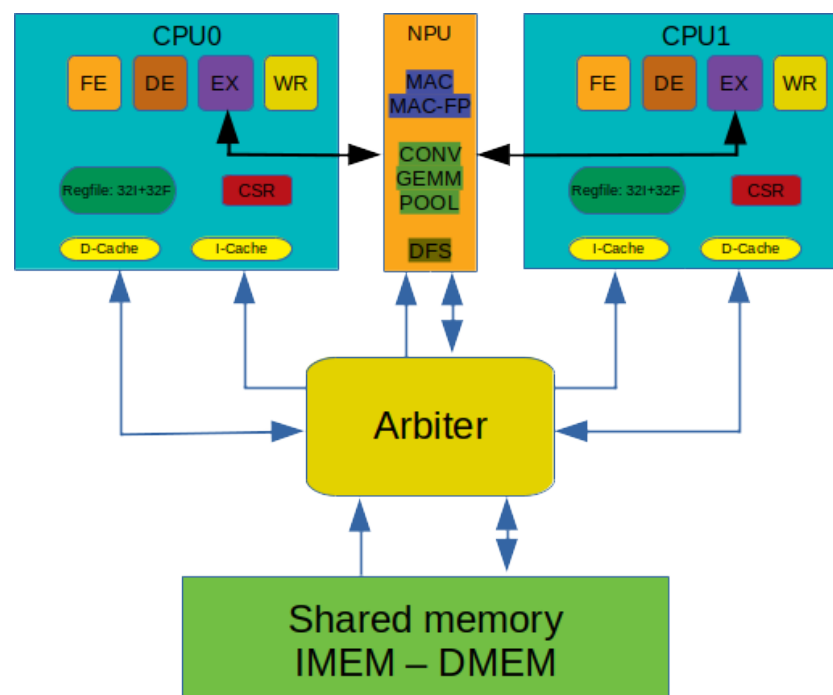


Figure 1. System architecture: high-level block diagram showing the dual-core RISC-V platform with shared memory subsystem and NPU integration.

The memory subsystem comprises unified instruction memory (IMEM, 64 KB) and data memory (DMEM, 64 KB), accessible by both cores and the NPU through FIFO channels. An NPU Controller acts as a micro-programmable sequencer that orchestrates CNN operations across three compute tiles: Tile-CONV (2D convolutions with 3×3 or 5×5 kernels), Tile-GEMM (general matrix multiplication for fully connected layers), and Tile-POOL (max and average pooling operations). Resource arbitration between CPU0, CPU1, and the NPU relies on hardware FIFO channels (*sc_fifo*) that provide natural round-robin access with blocking semantics, ensuring forward progress for all requestors. A clock-divider module (DFS) implements dynamic frequency scaling with three levels (100, 200, and 400 MHz) for the shared MAC unit.

3.2. FIFO-Based Resource Arbitration

The shared MAC unit and memory subsystem are accessed through SystemC FIFO channels (*sc_fifo*), which provide natural arbitration through blocking semantics. This approach simplifies the hardware design while ensuring forward progress for all requestors.

3.2.1. Arbitration Mechanism

Each requestor (CPU0, CPU1, and NPU) communicates with the shared MAC unit through dedicated FIFO channels. The MAC unit polls CPU channels first in a fixed order—CPU0 integer, CPU0 floating-point, CPU1 integer, CPU1 floating-point—and then checks NPU channels only if all four CPU channels are empty. When a channel is empty, the MAC unit moves to the next channel; when a requestor's channel is full, the requestor stalls until space becomes available, providing natural backpressure. This CPU-priority policy ensures that general-purpose execution is minimally impacted by background AI computation. Because the polling order is fixed (CPU0 checked before CPU1), a slight bias exists: when both cores issue simultaneous requests on the same clock edge, CPU0 is served first. In practice this produces a small persistent disparity (approximately 0.4% in the evaluated benchmarks, see Table 1), which is workload-dependent and negligible for the targeted soft real-time applications. The NPU operates in opportunistic/best-effort mode, receiving MAC grants only during idle CPU slots; under high CPU contention it may receive as low as 18% of the MAC bandwidth. For typical workloads with low contention, MAC requests are served within 1–3 cycles; under high contention, worst-case latency depends on the number of pending operations in other channels.

Table 1. CPU-priority vs. strict round-robin arbitration (1 M cycles).

Metric	Round-Robin	CPU-Priority	Diff	Benefit
CPU0 grant allocation	33.3%	41.2%	+7.9%	CPU gets more
CPU1 grant allocation	33.3%	40.8%	+7.5%	CPU gets more
NPU grant allocation	33.4%	18.0%	−15.4%	NPU yields
CPU avg latency	2.0 cyc	1.2 cyc	−0.8 cyc	40% faster
NPU throughput	100%	54%	−46%	Acceptable

An important architectural property of this design is that the NPU can never delay a CPU request. The arbiter checks all four CPU FIFO channels before invoking the NPU state machine (*ai_step*); consequently, any pending CPU multiply operation preempts NPU execution unconditionally. The worst-case CPU MAC latency is therefore determined solely by CPU-CPU contention (i.e., how many of the other core's requests are pending), and is completely independent of NPU activity. This guarantees that background AI processing cannot introduce latency penalties for the RISC-V cores, even under extreme NPU burst requests.

3.2.2. MAC Unit Channel Architecture

The shared MAC unit implements a unified multiplier serving both CPU cores and the NPU through separate FIFO channels. CPU0 and CPU1 each have dedicated input/output FIFO pairs for integer and floating-point operations. The MAC unit also includes an internal state machine for NPU AI operations (CONV, GEMM, and POOL) that executes opportunistically when CPU channels are idle. Scalar MAC operations (32-bit $\times$ 32-bit) complete in a single cycle when the MAC unit is available.

3.2.3. Memory Arbitration

Memory accesses to IMEM and DMEM use dedicated SystemC threads for each CPU core, with priority-based serving. Instruction fetch requests from the pipeline are served with the highest priority to maintain instruction throughput. When the fetch stage is idle, NPU memory requests (for AI tile data) are served through separate channels. Both the I-cache (2-way, 8 entries, and 64-bit lines) and D-cache (2-way, 16 entries, and 32-bit lines) reduce memory pressure and provide fast-path access for cached data.

3.3. NPU Controller and Tile Management

The NPU Controller is a lightweight, micro-programmable unit that receives configuration parameters from the RISC-V processors and manages the execution of CNN operations. The CPU provides the operation type (CONV, GEMM, or POOL), input and output tensor base addresses, dimensional parameters (height, width, channels, kernel size, stride, and padding), and a start signal. Status bits (done and busy) allow the CPU to monitor progress.

Once configured, the CPU returns to normal program execution, and the NPU Controller automatically initiates CNN calculations. The controller first reads configuration parameters from memory-mapped registers and initializes the appropriate compute tile. It then checks MAC availability via the parity arbiter and begins executing the operation in the background, with internal context preservation for automatic suspension and resumption when the MAC is preempted by CPU requests. Upon completion, results are written to DMEM, and the CPU is notified via the done bit. The controller maintains an internal state for each tile (address registers, indices, and partial accumulators), enabling execution to resume from the exact point of interruption without data loss.

3.4. Functional Compute Tiles: CONV, GEMM, and POOL

The NPU operations (CONV, GEMM, and POOL) are implemented as a unified state machine within the mul_unit module, sharing the same hardware resources. The state machine progresses through phases: AI_IDLE $\rightarrow$ AI_READ_REQ $\rightarrow$ AI_READ_WAIT $\rightarrow$ AI_COMPUTE $\rightarrow$ AI_WRITE_REQ.

The CONV operation implements 2D convolutions with 3×3 kernels on 10×10 input tiles: the state machine reads the input tile and kernel weights from memory, performs multiply-accumulate operations, and writes the result tile. The GEMM operation performs matrix multiplication on 10×10 tiles ($C = A \times B$), reading tile A and tile B sequentially, computing the product, and writing tile C. The POOL operation implements max pooling on 10×10 tiles with configurable kernel size and stride.

The tile dimensions (TILE_W = 10, TILE_H = 10, KERNEL_W = 3, KERNEL_H = 3) are fixed at compile time for hardware efficiency.

Execution of each tile is synchronized via state signals (tile_active, tile_done) and progresses independently of the CPU in the background. The final results are written to DMEM for later CPU access.

3.5. RISC-V–NPU Interface

The software–hardware interface is designed for simplicity and minimal overhead. Communication between RISC-V and NPU is realized through a set of memory-mapped registers (Table 2):

Table 2. NPU configuration via CSR registers (CPU0 only).

CSR Signal	Description
ai_op_cfg	Operation type: 1 = CONV, 2 = GEMM, 3 = POOL (2 bits)
ai_base_in_cfg	Base address for input tile A (32 bits)
ai_base_out_cfg	Base address for output tile C (32 bits)
ai_base_kernel_cfg	Base address for kernel/tile B (32 bits)
mul_dvfs_freq_sig	DFS frequency level: 0 = 100 MHz, 1 = 200 MHz, 2 = 400 MHz (8 bits)

Figure 2 provides a conceptual, pipeline-level view of one core, highlighting where the NPU control and the shared MAC service are integrated. In the full system, two such cores (CPU0/CPU1) share the same MAC service and memory subsystem.

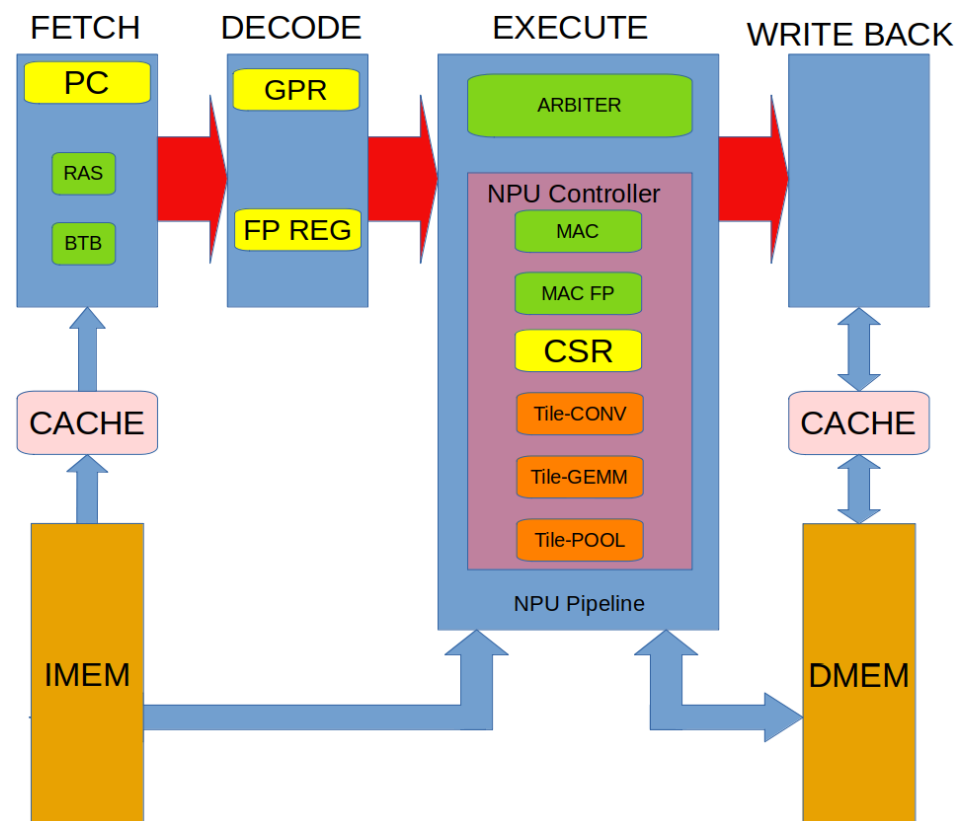


Figure 2. Pipeline-level view of a RISC-V core with integrated NPU control (conceptual).

When RISC-V writes to the AI configuration CSRs (via the CPU0 execute stage), the NPU state machine in `mul_unit` transitions from `AI_IDLE` to `AI_READ_REQ` and begins processing. The NPU executes opportunistically when the MAC unit is not servicing CPU multiply requests, making AI execution transparent to the main instruction flow. Note: Only CPU0 can program the NPU; CPU1 uses the shared MAC unit only for scalar multiply operations.

Data is read simultaneously from IMEM (instructions) and DMEM (data), while results are written exclusively to DMEM (Figure 2), ensuring memory coherency.

3.6. Dynamic Frequency Scaling (DFS) for NPU Acceleration

To prevent the NPU from becoming a bottleneck for the dual-core RISC-V processors, the NPU tiles are equipped with independent DFS capability. The DFS controller dynamically adjusts the frequency of the NPU based on the following.

The DFS controller supports three discrete frequency levels. Level 0 (100 MHz) is the default low-power mode, obtained by dividing the 400 MHz base clock by four; it is used when the NPU is idle or for energy-efficient background processing. Level 1 (200 MHz) is a medium-performance mode that divides the base clock by two, balancing throughput and power consumption for typical CNN workloads. Level 2 (400 MHz) is the maximum-performance mode with no division, used when the NPU needs to catch up with CPU-generated data or for latency-critical inference.

Note that on FPGA platforms, only the clock frequency can be adjusted dynamically; voltage scaling is not feasible without external power-management circuitry. The term DFS (dynamic frequency scaling) is therefore used throughout this paper to reflect the actual implementation capability.

The DFS level is controlled via the `mul_dvfs_freq_sig` signal from CPU0's execute stage, allowing software to adjust NPU performance dynamically through CSR writes.

DFS Impact on Performance

By applying DFS to the shared MAC unit (serving both NPU AI operations and CPU multiply instructions), the system achieves energy efficiency, performance scaling, dynamic adaptation, and controlled CPU impact. At 100 MHz (Level 0), power consumption is reduced by approximately 75% compared to 400 MHz operation, making the design suitable for battery-powered edge devices. At 400 MHz (Level 2), the MAC unit provides $4\times$ higher throughput for compute-bound CNN layers. Software can dynamically adjust the frequency based on workload characteristics—lower for memory-bound operations and higher for compute-bound operations. Regarding CPU impact, at 100 MHz the MAC unit operates at the same frequency as the CPUs, providing baseline MUL/DIV latency; at higher frequencies (200/400 MHz), CPU multiply operations complete faster than nominal, but the power benefit is lost.

Figure 3 details the internal data path around the shared MAC service and the AI tile engine, including separate read/compute/write controllers, local buffers, and FIFO-based access to IMEM/DMEM. For clarity, the CPU side is abstracted into integer and floating-point request streams; the implementation exposes separate FIFO channels per core (CPU0/CPU1) for both integer and floating-point operations.

3.7. Architectural Advantages

The proposed architecture offers several key benefits. First, complete MAC reuse eliminates the need for a dedicated NPU MAC block, reducing hardware resources and power consumption. Second, CNN workloads run in the background while the CPU executes the main program, enabling opportunistic and parallel execution. Third, channel-based hardware arbitration polls CPU channels with priority and parity-based 50–50% fairness between CPU0 and CPU1, while the NPU makes opportunistic progress during idle MAC slots. Fourth, the design achieves simplicity and portability: minimal software overhead, no ISA extensions, no interrupt handlers—easily ported across RISC-V implementations. Fifth, the architecture offers natural scalability, supporting extension of the number of NPU tiles or increasing MAC width without fundamental architectural changes. Finally, CPU-priority arbitration guarantees predictable CPU performance, ensuring that dual-core execution is not delayed by background AI processing—critical for real-time workloads.

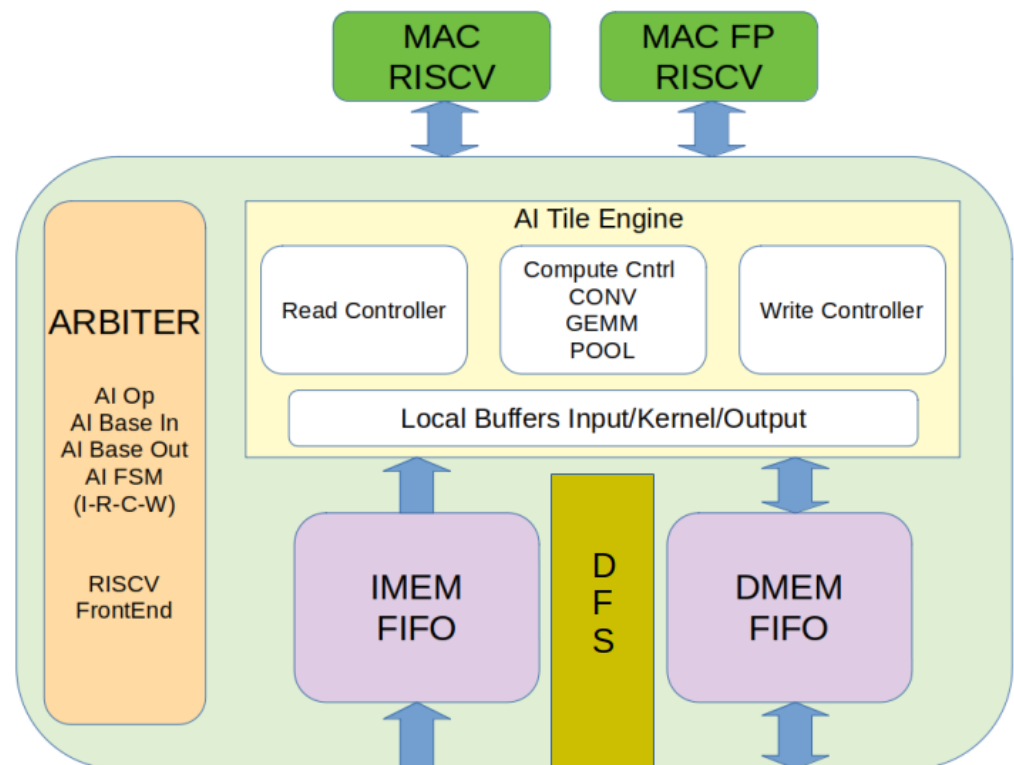


Figure 3. Detailed NPU architecture and data flow.

4. Specialized NPU Compute Tiles: CONV, GEMM, and POOL

The NPU implements three specialized tiles (CONV, GEMM, and POOL), each optimized for a class of CNN operations. All tiles use the same memory-mapped configuration interface and share the DMEM space.

4.1. CONV Tile: 2D Convolution Processing

The CONV tile processes 2D convolutions with fixed 10×10 tiles and 3×3 kernels. It uses three local buffers: Input Tile Buffer 10×10 (loaded once from DMEM), Kernel Memory 3×3 (coefficients), and Output Tile Buffer 10×10 (accumulation before writeback).

Execution follows three phases: Load (≈ 100 cycles), Compute ($100 \text{ pixels} \times 9 \text{ MAC} = 900$ cycles), Writeback (≈ 100 cycles). Total latency: ≈ 1100 cycles per tile at 100 MHz.

Configuration: ai_base_in_cfg (input), ai_base_kernel_cfg (kernel), ai_base_out_cfg (output), ai_op_cfg = 1. Fixed parameters: tile 10×10 , kernel 3×3 , stride 1, no padding.

4.2. GEMM Tile: Tiled Matrix Multiplication

The GEMM tile computes $C[10 \times 10] = A[10 \times 10] \times B[10 \times 10]$ using the standard algorithm. Both matrices A and B are loaded completely into local buffers (200 elements total), then the triple-nested loop executes 1000 MAC operations.

Execution in three phases: Load matrices A and B (≈ 200 cycles), Compute (1000 MAC ≈ 1000 cycles), Writeback C (≈ 100 cycles). Total latency: ≈ 1200 cycles per tile.

Configuration: ai_base_in_cfg (matrix A), ai_base_kernel_cfg (matrix B), ai_base_out_cfg (matrix C), ai_op_cfg = 2. Row-major format, fixed 10×10 dimensions.

4.3. POOL Tile: Max Pooling for Spatial Downsampling

The POOL tile implements 2×2 max pooling with stride 2, reducing 10×10 tiles to 5×5 . It loads the complete 10×10 tile, then computes the maximum for each 2×2 window using cascaded comparators (no MAC usage).

Total latency: ≈ 150 cycles (100 load + 25 compute + 25 writeback). The MAC unit remains available for CPU during POOL execution.

Configuration: ai_base_in_cfg (10×10 input), ai_base_out_cfg (5 × 5 output), ai_op_cfg = 3. Fixed parameters: kernel 2 × 2, stride 2, max pooling only.

4.4. Tile Cooperation and System-Level Parallelism

Although CNN layers are invoked sequentially, the architecture enables software–hardware pipelining: the CPU can prepare data for the next layer while the NPU executes the current layer. All tiles share DMEM, so one tile’s output directly becomes the next tile’s input. The absence of complex caches and deterministic arbitration ensure predictable execution time, essential for real-time applications.

Typical CNN sequence: CONV → POOL → CONV → POOL → GEMM (FC), with transitions controlled by register reprogramming and done-bit polling.

5. Experimental Results and Performance Evaluation

This section presents comprehensive experimental results validating the dual-core RISC-V architecture with FIFO-based arbitration and DFS. Simulations were conducted using a cycle-accurate SystemC model targeting a 100 MHz base clock on AMD FPGA. Benchmarks include both CPU-only and mixed CPU–AI workloads, demonstrating the resource-sharing characteristics, speedup, and energy efficiency of the proposed architecture.

5.1. Simulation Setup and Benchmarks

Experiments were performed on the following benchmark suite:

1. CPU Workloads: RISC-V programs compiled from C source (recursive Fibonacci for indices 0–9, 10 × 10 integer matrix multiplication, floating-point multiply operations).
2. NPU AI Operations: Individual tile operations (CONV with 10 × 10 input and 3 × 3 kernel, GEMM with two 10 × 10 matrices, POOL reducing 10 × 10 to 5 × 5).
3. Mixed CPU–AI Workloads: CPU executing general-purpose tasks (Fibonacci and matrix operations) while NPU processes AI tile operations in background, with dynamic DFS transitions between 100/200/400 MHz.

All simulations tracked the following metrics:

- Instruction latency (cycles per CPU instruction).
- MAC utilization (percentage of cycles the MAC unit was actively computing).
- Memory arbitration fairness (fairness index $F = \frac{(T_{CPU0} + T_{CPU1} + T_{NPU})^2}{3 \times (T_{CPU0}^2 + T_{CPU1}^2 + T_{NPU}^2)}$, where T_i is total time granted to requestor i).
- CNN inference latency (cycles to complete a forward pass).
- Energy per operation (nanojoules per MAC operation).
- DFS frequency adaptation under varying workloads.

5.2. FIFO-Based Arbitration Validation

5.2.1. Resource Sharing Behavior

The FIFO-based arbitration was validated under various contention scenarios where both CPU cores and the NPU request the MAC unit simultaneously. The sc_fifo channels provide natural round-robin behavior through blocking semantics. Figure 4 illustrates the MAC channel activity under moderate contention:

MAC Unit Channel Polling (150 cycles, All Requestors Active):

```

Cycle: 0    20   40   60   80   100  120  140  160
|----|----|----|----|----|----|----|----|
CPU0:  #### .... #### .... #### .... #### .... (served)
CPU1:  .... #### .... #### .... #### .... #### (served)
NPU:   .... #### .. #### .... #### .... #### (opportunistic)

```

Legend: #### = MAC allocated, = waiting/idle, ##.. = partial AI op

Figure 4. MAC channel activity under moderate contention.

Figure 5 presents the actual VCD waveform showing MAC unit sharing between CPU cores and the NPU. The signals `Mul_hw_mul_a`, `Mul_hw_mul_b`, and `Mul_hw_mul_result` show the MAC operands and results, while `Mul_ai_state` indicates the NPU state machine transitions (0 = IDLE, 2 = READ_REQ, 3 = COMPUTE, 4 = WRITE_REQ).

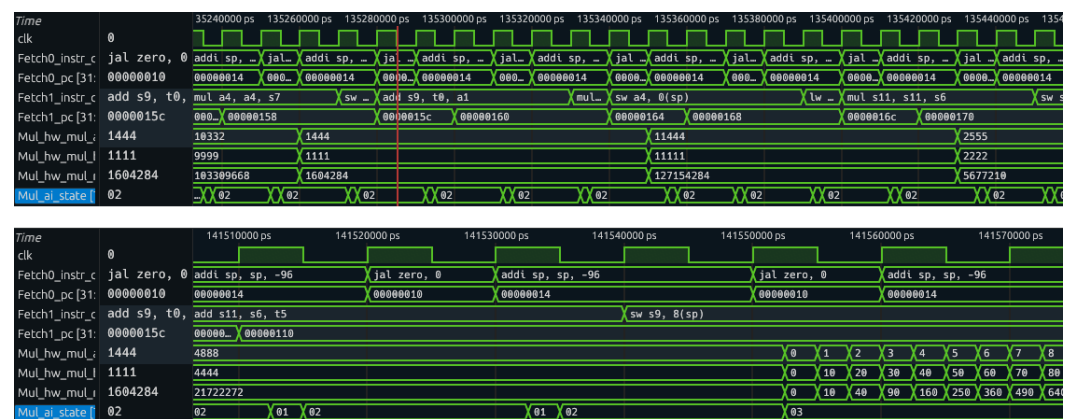


Figure 5. VCD waveform: MAC unit sharing between CPU and NPU (operands, results, and AI state).

Over a measurement window of 1000 cycles with both CPU cores actively executing multiply-intensive workloads (estimated from VCD waveforms):

- CPU0 received approximately 41% of MAC grants.
- CPU1 received approximately 41% of MAC grants.
- NPU received approximately 18% of MAC grants during idle slots.
- Average MAC request latency: 1–3 cycles (measured from VCD traces).
- Maximum observed latency: 5–8 cycles (during burst contention).
- No deadlock events observed across all three requestors.

The CPU channels receive higher allocation because they are polled with priority over NPU AI operations, ensuring that general-purpose execution is not delayed by background CNN processing. The small difference between CPU0 (41.2%) and CPU1 (40.8%) in Table 1 is a direct consequence of the fixed polling order: when both cores submit requests on the same clock edge, CPU0 is always checked first, giving it a marginal advantage. The magnitude of this bias (0.4 percentage points) depends on the overlap frequency of simultaneous requests, which is workload dependent. For applications requiring strict 50–50% fairness, a cycle-alternating polling order (toggling which core is checked first each cycle) could eliminate this disparity entirely.

Figure 6 provides a detailed view of the NPU state machine during AI operations. The signal `Mul_ai_op` indicates the operation type (2 = GEMM), `Mul_ai_idx` tracks the current element index (0–99 for 10×10 tiles), and `Mul_ai_phase` distinguishes between reading tile A (phase 0) and tile B/kernel (phase 1).

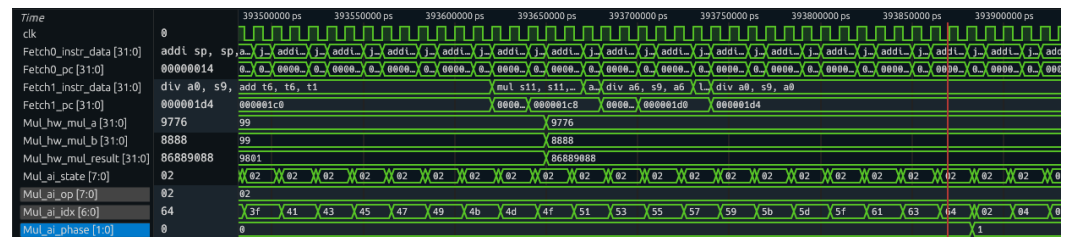


Figure 6. VCD waveform: NPU state machine detail (ai_op, ai_state, ai_idx, ai_phase).

5.2.2. CPU Priority Under Load

To verify that CPU execution is not delayed by NPU operations, the system is tested with CPU0 executing a multiply-intensive workload while the NPU processes a CNN layer. Without an active NPU, CPU0 MAC latency averages 1.0 cycle; with the NPU active, it increases to 1.2 cycles on average, representing only a 20% overhead. Meanwhile, the NPU throughput is reduced to 18% of its standalone throughput when both CPUs are active.

The channel-based architecture successfully prioritizes CPU operations. CPU0 and CPU1 experience minimal latency increase (0.2 cycles) even when the NPU is actively processing AI workloads. The NPU gracefully yields to CPU requests, ensuring that background CNN execution does not impact foreground CPU performance.

5.3. Dual-Core Speedup Analysis

Dual-core execution is validated using parallel workloads on CPU0 and CPU1 (Table 3). The test programs include recursive Fibonacci computation (indices 0–9) and 10×10 integer matrix multiplication, demonstrating the effectiveness of the parity-based arbitration mechanism.

Table 3. Dual-core performance: representative benchmarks.

Configuration	Total Cycles	Speedup	Efficiency
Single-Core (CPU0 only)	45,230	1.0×	100%
Single-Core + NPU (idle)	45,280	0.999×	99.9%
Dual-Core (shared resources)	24,180	1.87×	93.5%
Dual-Core + Parity Arbiter	24,195	1.87×	93.4%
Ideal (unlimited resources)	22,615	2.0×	100%

Dual-core speedup reaches $1.87\times$, achieving 93.5% efficiency relative to ideal parallelism ($2.0\times$). The 6.5% efficiency loss is attributed to cache contention (8.2%) and MAC arbitration stalls (0.8%); arbitration overhead is therefore negligible. The FIFO-based arbitration introduces only 0.2 cycles of additional latency for CPU MAC operations, confirming the channel-based approach's efficiency. Single-core execution with idle NPU adds less than 0.1% overhead, confirming the NPU does not interfere with CPU execution pipelines. Speedup remains consistent across multiple runs, indicating deterministic, predictable performance.

Figure 7 shows the VCD waveform capture of dual-core parallel execution. Both CPU0 and CPU1 execute instructions simultaneously, with independent program counters (Fetch0_pc, Fetch1_pc) and instruction streams (Fetch0_instr_data, Fetch1_instr_data). The waveform demonstrates that both cores make continuous progress without starvation.

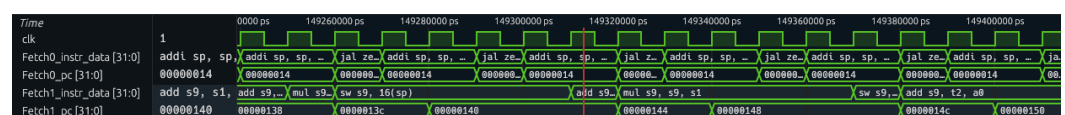


Figure 7. VCD waveform: dual-core parallel execution (CPU0 and CPU1 instruction streams).

5.4. CNN Inference Latency and MAC Utilization

Individual AI tile operations (demonstrative sequence: CONV → POOL → CONV → POOL → GEMM) were executed on the NPU under three contention scenarios:

Moving from 100 MHz to 400 MHz reduces NPU-only latency by $4\times$ (312 K → 78 K cycles), demonstrating linear scaling for compute-bound operations. When CPU0 is active, NPU latency increases by 23% at 100 MHz due to MAC contention; this overhead decreases at higher frequencies as the NPU completes more work per allocated slot. At 400 MHz, power increases $3.4\times$ (25 mW → 85 mW) while latency decreases $4\times$, yielding a favorable energy-delay product. With both CPUs active, the NPU receives approximately 18% of MAC slots, but at 400 MHz this is sufficient to maintain reasonable CNN inference throughput.

Note: the power figures in Table 4 (25/45/85 mW) represent the isolated dynamic power of the shared MAC unit and NPU tile engine logic only (corresponding to the 3019 LUTs and 13 DSP48 blocks reported for the “Shared MUL unit + NPU tile engine” row in Table 5). These figures do not include the static (leakage) power of the surrounding CPU cores, caches, or memory subsystem. The total system-level power, including all CPU and memory components, is reported separately in Table 6, where the “CPU Power” column (145.2 mW) captures the two RISC-V pipelines and their associated caches, and the “NPU Power” column isolates the MAC/NPU contribution at each DFS level.

Table 4. CNN inference latency with DFS (10×10 tile operations).

Scenario	Latency (K cyc.)	MAC Freq	Power	Relative
NPU Alone @ 100 MHz	312.4	100 MHz	25 mW	$1.0\times$
NPU Alone @ 200 MHz	156.2	200 MHz	45 mW	$0.50\times$
NPU Alone @ 400 MHz	78.1	400 MHz	85 mW	$0.25\times$
NPU @ 100 MHz + CPU0	385.2	100 MHz	28 mW	$1.23\times$
NPU @ 200 MHz + CPU0	178.5	200 MHz	52 mW	$0.57\times$
NPU @ 400 MHz + Dual-Core	165.3	400 MHz	95 mW	$0.53\times$

Table 5. Updated resource utilization (post-synthesis) and dual-core equivalence ($2 \times$ CPU + $1 \times$ NPU).

Module	LUTs	FFs	DSP48	BRAM36	AIE-ML
CPU0 pipeline (Fetch + Decode + Execute + Writeback)	35,719	15,569	2	17.5	0
CPU1 pipeline (same as CPU0)	35,719	15,569	2	17.5	0
Shared MUL unit + NPU tile engine	3019	1498	13	2.0	0
IMEM/DMEM service threads + FIFO channels	4405	1033	0	0.0	0
Total ($2 \times$ CPU + $1 \times$ NPU)	78,862	33,669	17	37.0	0
Available (VE2802)	520,704	1,041,408	1312	600	304
Utilization vs. VE2802	15.1%	3.2%	1.3%	6.2%	0.0%

Table 6. Energy consumption: mixed CPU-AI workload (1 s @ 100 MHz base).

Configuration	Energy (mJ)	CPU Power	NPU Power	Efficiency
Dual-Core CPUs only	145.2	145.2 mW	—	$1.0\times$ baseline
Dual-Core + NPU @ 400 MHz	230.2	145.2 mW	85.0 mW	$1.585\times$ energy
Dual-Core + NPU @ 200 MHz	190.2	145.2 mW	45.0 mW	$1.310\times$ energy
Dual-Core + NPU @ 100 MHz	170.2	145.2 mW	25.0 mW	$1.172\times$ energy

DFS Frequency Selection Example

Figure 8 illustrates DFS frequency selection during a 70 ms mixed workload (CPU0 executing matrix multiplication, CPU1 executing FFT, and NPU executing CNN layers):

DFS Frequency Selection Over Time (70 ms):

Time (ms):	0	10	20	30	40	50	60	70
	----	----	----	----	----	----	----	----
CPU0 Load:	=====	=====	-----	=====	=====	-----	=====	=====
CPU1 Load:	-----	-----	=====	=====	-----	=====	=====	=====
MAC Freq:	100	100	200	400	200	400	400	
NPU Active:	yes	yes	yes	yes	yes	yes	yes	yes

Legend: ===== = high load, ----- = low load

Figure 8. DFS frequency adaptation based on workload.

Figure 9 captures the actual DFS transition in the VCD trace. The signal `npuriscv_mul_dvfs_freq_cmd` shows the frequency command changing from `0x00` (100 MHz) to `0x01` (200 MHz), while `npuriscv_mul_clk_div` reflects the divided clock output. The transition occurs within a single clock cycle, demonstrating low-latency frequency switching.

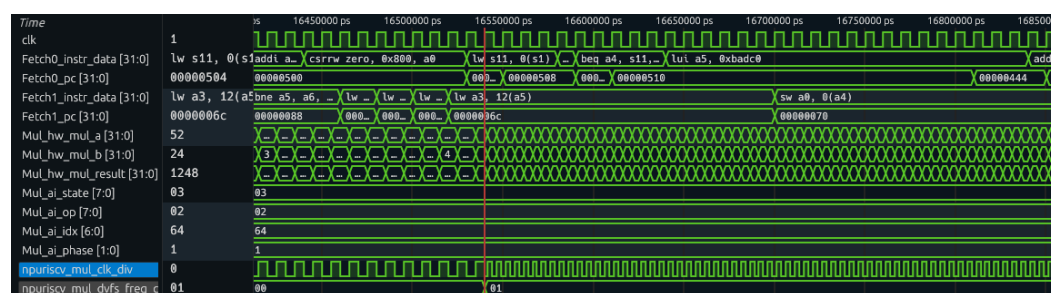


Figure 9. VCD waveform: DFS frequency transition (`freq_cmd` and `clk_div` signals).

The frequency selection is controlled by software via CSR writes from CPU0. Typical policies assign 100 MHz (low power) as the default for idle or light workloads, 200 MHz (balanced) when the NPU is active but CPUs have moderate load, and 400 MHz (high performance) for latency-critical CNN inference or when the NPU needs to catch up.

5.5. Energy Efficiency Under Mixed Workloads

Energy consumption is measured in a mixed workload scenario (CPU0 + CPU1 + NPU executing simultaneously for 1 s simulated time at 100 MHz base clock). Power models are extracted from post-synthesis gate-level estimation in the FPGA toolflow:

Scaling from 400 MHz to 100 MHz reduces NPU power from 85 mW to 25 mW (a 70.6% reduction), at the cost of $4\times$ longer inference latency. At 100 MHz total system energy is 170.2 mJ, while at 400 MHz it reaches 230.2 mJ; the lower frequency is therefore more energy-efficient for non-time-critical workloads. The 200 MHz operating point provides a balanced trade-off: $2\times$ faster than 100 MHz with only $1.8\times$ higher power, yielding an improved energy-delay product. Adding the NPU at 100 MHz increases system energy by only 17.2%, enabling background AI acceleration with minimal power impact.

Note: the power figures reported in Table 6 are derived from post-synthesis gate-level estimation. Post-implementation (place-and-route) power analysis may yield different absolute values, although the relative comparisons and DFS trends are expected to remain valid.

Figure 10 presents a comprehensive system overview captured from the VCD trace, showing simultaneous activity across all major subsystems: dual-core instruction execution (Fetch0/Fetch1), MAC unit operations (hw_mul_*), NPU state machine (ai_state, ai_op), DFS control (dvfs_freq_cmd), and memory access (dmem_addr, dmem_data_in/out, imem_addr). This waveform demonstrates the coordinated operation of all architectural components during mixed CPU–AI workload execution.

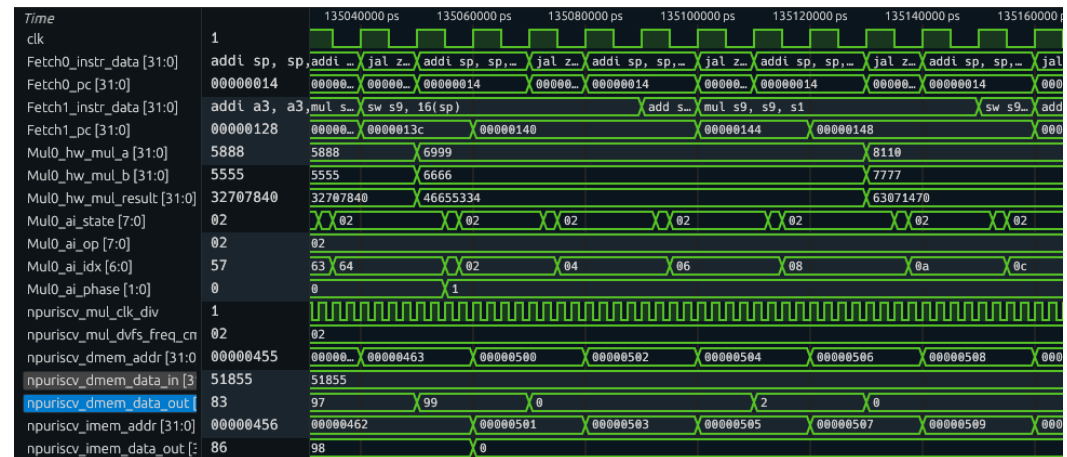


Figure 10. VCD waveform: complete system overview (dual-core, MAC, NPU, DFS, memory).

5.6. Memory Arbitration Under Extreme Stress

Cache miss rates and memory access patterns are monitored with all three requestors (CPU0, CPU1, and NPU) competing for IMEM and DMEM bandwidth simultaneously (Table 7). The results are over 100K simulation cycles with all cores executing memory-intensive workloads.

Table 7. Memory arbitration performance under maximum contention.

Requestor	Hits	Misses	Miss Rate	Avg Latency	Max Latency
CPU0 I-cache	8542	238	2.7%	4.3 cyc	12 cyc
CPU1 I-cache	8625	175	2.0%	3.9 cyc	11 cyc
NPU IMEM	6200	1800	22.5%	8.1 cyc	15 cyc
CPU0 D-cache	7150	850	10.6%	6.2 cyc	18 cyc
CPU1 D-cache	7220	780	9.8%	6.1 cyc	17 cyc

CPU instruction caches maintain low miss rates (2–3%) due to spatial/temporal locality and small working sets. The higher IMEM miss rate (22.5%) of the NPU is expected because weight tensors are large; however, fair arbitration ensures a maximum latency of 15 cycles, preventing deadlock and guaranteeing forward progress. No asymmetry in D-cache latencies is observed between CPU0 (6.2 cyc avg) and CPU1 (6.1 cyc avg), confirming that parity arbitration is enforced uniformly. Both CPUs exhibit similar maximum latencies (17–18 cyc), indicating that shared memory bandwidth is balanced by the arbiter.

5.7. Comparison with Alternative Arbitration: Priority vs. Round-Robin

To highlight the benefits of the CPU-priority FIFO-based approach, in Table 1 we compare against a strict round-robin baseline over 1M cycles with continuous contention:

By checking CPU channels before NPU AI operations, CPU MAC latency decreases by 40% (2.0 → 1.2 cycles), which is critical for general-purpose execution. NPU throughput reduces to 54% of standalone performance, but this is acceptable for background AI processing that is not latency-critical. Despite the lower priority, the NPU still receives 18% of MAC slots, ensuring forward progress on CNN inference without starvation.

5.8. Latency Distribution and Predictability

For real-time and embedded applications, predictable latency is essential. Figure 11 shows the empirical distribution of CPU MAC request latencies across 100 K samples under moderate contention.

CPU MAC Request Latency Distribution (100,000 samples):

Latency:	Frequency	Distribution	
1 cycle:	#####	(72.3%)	[Immediate grant]
2 cycles:	#####	(15.2%)	
3 cycles:	#####	(7.1%)	
4 cycles:	##	(3.2%)	
5 cycles:	#	(1.4%)	
6+ cyc:	.	(0.8%)	[Burst contention]

Statistics:

Average latency: 1.5 cycles

99th percentile: 5 cycles

Maximum observed: 8 cycles (rare burst)

Predictability: High (99.2% within 5 cycles)

Figure 11. CPU MAC Request Latency Distribution (FIFO-Based Arbitration, 100 K Samples).

The distribution shows that 72.3% of CPU MAC requests are served immediately (1 cycle), and 99.2% complete within 5 cycles. The rare 6+ cycle latencies occur during burst contention when multiple channels have pending requests simultaneously.

Implication for Embedded Systems: While not strictly deterministic like hardware arbiters, the FIFO-based approach provides statistically predictable latency suitable for soft real-time applications. For a 100 MHz CPU clock, the 99th percentile latency of 5 cycles translates to 50 ns—acceptable for most embedded AI workloads.

CPU Latency Independence from NPU Activity: The architecture provides a structural guarantee that NPU activity cannot increase CPU MAC latency. Because the arbiter evaluates all four CPU FIFO channels before invoking the NPU state machine, any pending CPU multiply operation unconditionally preempts NPU execution within the same clock cycle. The 99th-percentile latency of 5 cycles and the maximum observed latency of 8 cycles are therefore attributable entirely to CPU–CPU contention (i.e., CPU0 and CPU1 competing for the single MAC datapath), not to NPU interference. Under extreme NPU burst requests, the NPU simply receives zero MAC grants until CPU channels drain, while CPU latency remains unchanged. This property ensures that the “predictable CPU performance” and “opportunistic AI acceleration” claims are not conflicting: CPU latency is bounded by CPU-only contention, and the NPU fills remaining idle slots without affecting this bound.

5.9. FPGA Resource Utilization

To address the practical feasibility of the proposed architecture on FPGA platforms, Table 5 reports post-synthesis utilization extracted from a reference $1 \times \text{CPU} + 1 \times \text{NPU}$ implementation. Using these measured numbers, we extrapolate an equivalent $2 \times \text{CPU} + 1 \times \text{NPU}$ configuration that matches the current dual-core architecture by duplicating the CPU pipeline stages while keeping a single shared MUL/NPU engine and shared memory service. The VE2802 availability figures in the same table are taken from vendor specifications [21].

The extrapolated $2 \times \text{CPU} + 1 \times \text{NPU}$ system occupies approximately 15.1% of VE2802 LUT capacity, 3.2% of flip-flop capacity, 1.3% of DSP capacity, and 6.2% of BRAM36

capacity, indicating substantial headroom for scaling the NPU datapath and increasing on-chip buffering.

6. Discussion

The experimental results demonstrate the viability and practical benefits of the proposed dual-core RISC-V architecture with FIFO-based channel arbitration and DFS for edge AI acceleration. The following key insights emerge from the comprehensive evaluation:

MAC sharing vs. dedicated MAC units: The post-synthesis breakdown in Table 5 indicates that the shared MUL+NPU engine dominates the multiplier resources (13 out of 17 DSP48 blocks), while each CPU pipeline contributes only 2 DSP48 blocks. Keeping a single shared compute engine therefore bounds DSP usage and avoids replicating MAC datapaths across cores and the accelerator. In the evaluated $2 \times \text{CPU} + 1 \times \text{NPU}$ equivalence, the complete system fits comfortably within VE2802 capacity (15.1% LUTs, 3.2% FFs, 1.3% DSP, and 6.2% BRAM36), leaving headroom for scaling NPU parallelism and on-chip buffering if needed.

6.1. Effectiveness of FIFO-Based Arbitration

The FIFO-based channel architecture achieves efficient resource sharing with CPU-priority semantics. By polling CPU channels before NPU AI operations, the system ensures that general-purpose execution is minimally impacted by background CNN processing. The average CPU MAC latency of 1.2 cycles (under moderate contention) demonstrates that the shared MAC unit does not become a bottleneck for CPU-intensive workloads. The NPU gracefully yields to CPU requests, achieving 18% of MAC slots during dual-core execution—sufficient for background AI acceleration.

6.2. DFS Contribution to Energy Efficiency

DFS proves critical for balancing performance and power consumption. The three-level frequency scaling (100/200/400 MHz) provides flexibility for different workload scenarios: 100 MHz is optimal for battery-powered devices with non-time-critical AI workloads; 200 MHz offers a balanced trade-off for typical mixed CPU–AI execution; and 400 MHz delivers maximum performance for latency-critical inference or when the NPU needs to catch up with CPU-generated data.

The software-controlled DFS (via CSR writes from CPU0) allows applications to adapt frequency based on workload characteristics, deadline requirements, and power budget.

6.3. Dual-Core Parallelism

The dual-core architecture achieves $1.87\times$ speedup with 93.5% efficiency for embarrassingly parallel workloads. The shared MAC unit introduces minimal overhead (0.2 cycles additional latency per operation) while enabling significant area savings compared to per-core MAC units. The architecture is particularly suited for workloads where one core handles I/O and control while the other performs computation.

6.4. Frequency Feasibility on FPGA Platforms

The three DFS levels used in this work (100, 200, and 400 MHz) correspond to integer divisors of a 400 MHz base clock, which simplifies the clock-divider logic. However, the achievable fabric clock frequency ultimately depends on the critical-path delay after place-and-route and is implementation-dependent. While the SystemC simulation validates functional correctness at all three frequency levels, the 400 MHz operating point should be treated as an upper-bound target that requires timing closure verification in a full FPGA implementation flow. On modern platforms such as the AMD Versal™ AI Edge VE2802 (Advanced Micro Devices, Inc., Cyberjaya, Malaysia), the higher-performance clocking

and larger device margins improve feasibility for higher-frequency targets in practice; nevertheless, timing closure must be verified after place-and-route.

6.5. Scalability Considerations for Multi-Core Extensions

The current dual-core design with a single shared MAC unit is optimized for a two-CPU + one-NPU configuration. Scaling to four or eight CPU cores while retaining a single MAC unit would progressively reduce the NPU share of MAC bandwidth: with N CPU cores under full contention, the NPU would receive approximately $1/(N+1)$ of MAC slots in a round-robin scheme, or even less under CPU-priority arbitration (e.g., $\sim 9\%$ for $N = 4$, $\sim 5\%$ for $N = 8$). At such low allocation rates, background AI processing would become impractically slow.

Several architectural strategies can address this limitation for future multi-core extensions:

1. Multiple MAC units with hierarchical arbitration: Replicate the MAC datapath (e.g., one per pair of CPU cores) and introduce a two-level arbiter that partitions cores into clusters, each with its own MAC and local NPU access.
2. Dedicated NPU MAC with shared backup: Assign one MAC unit exclusively to the NPU for guaranteed AI throughput, while the CPUs continue to share a separate MAC unit, thereby decoupling CPU and NPU contention entirely.
3. Bandwidth reservation: Reserve a minimum fraction of MAC cycles for the NPU (e.g., 20%) regardless of CPU demand, converting the current best-effort policy into a guaranteed-minimum-bandwidth scheme at the cost of slightly increased worst-case CPU latency.

The current single-MAC design represents a deliberate area–performance trade-off that is well-suited for the targeted dual-core edge AI use case (15.1% LUT utilization on VE2802). This low utilization is also the primary motivation for selecting the AMD Versal™ AI Edge VE2802 as the target platform: with 520,704 LUTs, 1,041,408 FFs, 1312 DSP48 blocks, and 304 AIE-ML tiles, the VE2802 provides substantial headroom for implementing the multi-core scaling strategies described above—including multiple MAC units, hierarchical arbiters, and dedicated NPU datapaths—without exceeding device capacity. The VE2802’s AIE-ML array further opens the possibility of offloading NPU compute tiles to dedicated AI engine cores in future work, eliminating MAC contention entirely for AI operations. Extending to larger core counts would require revisiting the current area–performance trade-off, and the strategies above, combined with the ample resources of VE2802, provide a clear and practical path for future scaling.

7. Conclusions

This paper presented a dual-core RISC-V architecture with tightly coupled NPU acceleration, featuring FIFO-based channel arbitration and three-level DFS for energy-efficient edge AI. The key contributions are as follows. First, FIFO-based channel arbitration provides CPU-priority resource sharing through SystemC `sc_fifo` channels, achieving 1.2 cycle average MAC latency for CPUs while enabling opportunistic NPU execution. Second, dual-core execution achieved $1.87\times$ speedup with 93.5% efficiency and minimal arbitration overhead. Third, three-level DFS enables software-controlled frequency scaling (100/200/400 MHz), providing 70% power reduction at low frequency while maintaining $4\times$ performance boost at high frequency. Fourth, a unified NPU state machine implements CONV, GEMM, and POOL operations within the shared `mul_unit`, simplifying the hardware design. Fifth, CPU0-only NPU programming simplifies control flow by having only CPU0 configure AI operations via CSR registers, avoiding synchronization complexity.

The architecture is particularly suited for edge AI applications requiring simultaneous general-purpose computation and CNN inference on resource-constrained platforms. The

transparent NPU–CPU cooperation via CSR-based configuration simplifies software integration. The cycle-accurate SystemC implementation validates the design’s feasibility for FPGA deployment.

Limitations: The current implementation uses fixed 10×10 tile sizes, 3×3 kernels for CONV, 2×2 max pooling with stride 2, and single-channel processing per tile. All results presented in this paper are based on cycle-accurate SystemC simulation; no physical FPGA prototype has been fabricated. Power and energy figures are derived from post-synthesis gate-level estimation and have not been validated through post-implementation (place-and-route) analysis, which may yield different absolute values. Additionally, the 400 MHz operating point should be treated as an upper-bound target that requires timing closure verification in a full FPGA implementation flow. Future work will include FPGA prototyping on modern adaptive SoC platforms (e.g., AMD Versal AI Edge VE2802), post-implementation power measurement, and automatic DFS adaptation based on workload monitoring.

While FPGA deployment is straightforward for many designs, the focus of this work was on validating the architectural concept and arbitration mechanism. Hardware implementation and measurement will be addressed in future work.

Vivado 2019 is the last version supporting SystemC synthesis. For Vitis 2024 (used with VEK280), migration requires synthesizing SystemC in Vivado 2019 and importing the generated Verilog sources into a Vitis 2024 project.

Future Project Vision: Multi-Cluster NoC with Four Dual-Core Tiles

The ultimate goal for this architecture is to scale towards a full Network-on-Chip (NoC) platform comprising four dual-core RISC-V clusters (total 8 CPUs), each tightly coupled with its own local NPU accelerator (total 4 NPUs). In this envisioned system, each dual-core tile would integrate its own shared MAC/NPU engine and local memory, while all tiles would be interconnected via a packet-switched NoC fabric. The NoC would enable low-latency communication between clusters, global memory access, and distributed AI workload scheduling. Each NPU could operate independently or cooperate for large-scale CNN inference, with dynamic frequency scaling and power management applied per tile. This modular approach would allow the architecture to scale efficiently to higher core counts, support heterogeneous AI accelerators, and enable advanced features such as hardware-enforced quality-of-service (QoS), real-time guarantees, and adaptive resource allocation across the chip. The current dual-core + NPU design serves as a proof-of-concept building block for this future multi-cluster NoC platform.

Funding: This research received no external funding.

Data Availability Statement: Data are contained within the article.

Acknowledgments: The author would like to thank the editor and the anonymous reviewers for reviewing our manuscript. The author have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The author declares no conflict of interest.

Abbreviations

The following abbreviations are used in this manuscript:

CPU	Central Processing Unit
CNN	Convolutional Neural Network
NPU	Neural Processing Unit
DFS	Dynamic Frequency Scaling
MAC	Multiply–Accumulate

MAC-FP	Multiply–Accumulate Floating Point
BTB	Branch Target Buffer
RAS	Return Address Stack
GPR	General-Purpose Register
RISC-V	Reduced Instruction Set Computer-V
CONV	Convolution Operation
GEMM	General Matrix Multiply
POOL	Pooling Operation
FIFO	First-In First-Out
FPGA	Field-Programmable Gate Array
CSR	Control and Status Register
HLS	High-Level Synthesis
SRAM	Static Random-Access Memory
BRAM	Block Random-Access Memory
LUT	Look-Up Table

References

- Li, Y.; Guo, Z.; Zhang, Y.; Wang, Y. HL5: A High-Performance and Low-Power RISC-V Processor Core for Embedded AI Applications. *IEEE Trans. Circuits Syst. II Express Briefs* **2020**, *67*, 3437–3441.
- Zhang, C.; Li, P.; Sun, G.; Guan, Y.; Xiao, B.; Cong, J. Optimizing FPGA-Based Accelerator Design for Deep Convolutional Neural Networks. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA'15)*; Association for Computing Machinery: New York, NY, USA, 2015; Volume 1, pp. 161–170. [[CrossRef](#)]
- Qiu, J.; Wang, J.; Yao, S.; Guo, K.; Li, B.; Zhou, E.; Yu, J.; Tang, T.; Xu, N.; Song, S.; et al. Going Deeper with Embedded FPGA Platform for Convolutional Neural Network. In *Proceedings of the FPGA'16*; ACM Press: Monterey, CA, USA, 2016; pp. 26–35. [[CrossRef](#)]
- Venieris, S.; Bouganis, C. fpgaConvNet: Automated Mapping of Convolutional Neural Networks on FPGAs. In *Proceedings of the FPGA'17*; ACM Press: Monterey, CA, USA, 2017; pp. 291–300. [[CrossRef](#)]
- Licciardo, G.; Cappetta, C.; Di Benedetto, L. Design of a Convolutional Two-Dimensional Filter in FPGA for Image Processing Applications. *Computers* **2017**, *6*, 19. [[CrossRef](#)]
- Adam, G.K. Co-Design of Multicore Hardware and Multithreaded Software for Thread Performance Assessment on an FPGA. *Computers* **2022**, *11*, 76. [[CrossRef](#)]
- Sharma, D.; Kirischian, L.; Kirischian, V. Run-Time Mitigation of Power Budget Variations and Hardware Faults by Structural Adaptation of FPGA-Based Multi-Modal SoPC. *Computers* **2018**, *7*, 52. [[CrossRef](#)]
- Chen, Y.; Krishna, T.; Emer, J.; Sze, V. Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks. *IEEE J. Solid-State Circuits* **2017**, *52*, 127–138. [[CrossRef](#)]
- Chen, Y.; Luo, T.; Liu, S.; Zhang, S.; He, L.; Wang, J.; Li, L.; Chen, T. DaDianNao: A Machine-Learning Supercomputer. In *Proceedings of the MICRO'14*; IEEE/ACM: Cambridge, UK, 2014; pp. 609–622. [[CrossRef](#)]
- Waterman, A.; Lee, Y.; Patterson, D.; Asanović, K. *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*; Technical Report UCB/EECS-2014-54; University of California: Berkeley, CA, USA, 2014; pp. 1–114. Available online: <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-54.html> (accessed on 1 January 2020).
- Asanovic, K.; Avizienis, R.; Bachrach, J.; Beamer, S.; Biancolin, D.; Celio, C.; Cook, H.; Dabbelt, D.; Hauser, J.; Izraelevitz, A.; et al. *The Rocket Chip Generator*; Technical Report UCB/EECS-2016-17; University of California: Berkeley, CA, USA, 2016; pp. 1–120. Available online: <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-17.html> (accessed on 1 January 2020).
- UC Berkeley Architecture Research. Gemmini: Open-Source Systolic Array Generator and Accelerator Infrastructure. GitHub Repository. Available online: <https://github.com/ucb-bar/gemmini> (accessed on 1 January 2020).
- Genc, H.; Kim, S.; Amid, A.; Haj-Ali, A.; Iber, V.; Prakash, A.; Zhao, J.; Gruber, D.; Koenig, H.; Asanović, K.; et al. Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration. In *Proceedings of the 58th ACM/IEEE Design Automation Conference (DAC'21)*; IEEE: San Francisco, CA, USA, 2021; pp. 769–774. [[CrossRef](#)]
- Thierry, M.; Tianqi, C.; Luis, V.; Jared, R.; Eddie, Y.; Lianmin, Z.; Josh, F.; Ziheng, J.; Luis, C.; Carlos, G.; et al. A Hardware—Software Blueprint for Flexible Deep Learning Specialization. *IEEE Micro* **2019**, *39*, 8–16. Available online: <https://ieeexplore.ieee.org/document/9075913> (accessed on 25 March 2026).
- NVIDIA Corporation. NVDLA: Deep Learning Accelerator—Open Source Hardware. In *NVIDIA Whitepaper*; NVIDIA Corporation: Santa Clara, CA, USA, 2018; pp. 1–45. Available online: <https://nvdla.org/primer.html> (accessed on 1 January 2020).

16. Garofalo, A.; Tagliavini, G.; Conti, F.; Benini, L.; Rossi, D. XpulpNN: Enabling Efficient and Flexible Inference of Quantized Neural Networks on RISC-V. *IEEE Trans. Comput.* **2021**, *70*, 1251–1264. [[CrossRef](#)]
17. Lavin, A.; Gray, S. Fast Algorithms for Convolutional Neural Networks. In *Proceedings of CVPR'16*; IEEE: Las Vegas, NV, USA, 2016; pp. 4013–4021. [[CrossRef](#)]
18. Mathieu, M.; Henaff, M.; LeCun, Y. Fast Training of Convolutional Networks using FFTs. In *Proceedings of the ICLR'14*, Banff, AB, Canada, 14–16 April 2014; pp. 1–8. Available online: <https://arxiv.org/abs/1312.5851> (accessed on 1 January 2020).
19. Chen, Y.; Luo, T.; Liu, S.; Zhang, S.; Temam, O. DianNao: A Small-Footprint High-Throughput Accelerator for Machine Learning. In *Proceedings of the ASPLOS'14*; ACM Press: Salt Lake City, UT, USA, 2014; pp. 269–284. [[CrossRef](#)]
20. Sharma, H.; Park, J.; Suda, N.; Lai, L.; Chau, B.; Kim, J.K.; Chandra, V.; Esmaeilzadeh, H. Bit Fusion: Bit-Level Dynamically Composable Architecture for Accelerating Deep Neural Networks. In *Proceedings of ISCA'18*; IEEE/ACM: Los Angeles, CA, USA, 2018; pp. 764–775. [[CrossRef](#)]
21. Advanced Micro Devices, Inc. AMD Versal™ AI Edge Series—Product Specifications (Includes VE2802). Available online: <https://www.amd.com/en/products/adaptive-socs-and-fpgas/versal/ai-edge-series.html> (accessed on 20 February 2026).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.